

First we'll define a quantum circuit that implements a query gate for one of the four functions f_1, f_2, f_3 or f_4 from one bit to one bit. As was discussed previously, the implementation of query gates is not really a part of Deutsch's algorithm itself — here we're essentially just showing one way to prepare the input (in the form of a circuit implementation of a query gate).

```
from qiskit import QuantumCircuit

def deutsch_function(case: int):
    """
    Generate a valid Deutsch function as a `QuantumCircuit`.
    """
    if case not in [1, 2, 3, 4]:
        raise ValueError("`case` must be 1, 2, 3, or 4.")

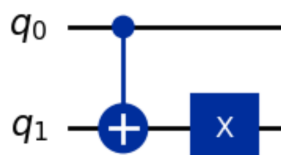
    f = QuantumCircuit(2)
    if case in [2, 3]:
        f.cx(0, 1)
    if case in [3, 4]:
        f.x(1)
    return f
```

✓ 2.0s

We can see what each circuit looks like using the draw method.

```
display(deutsch_function(3).draw(output='mpl'))
```

✓ 1.6s



Next we will create the actual quantum circuit for Deutsch's algorithm, substituting the query gate with a quantum circuit implementation given as an argument. Shortly we'll plug in one of the four circuits defined by the function `deutsch_function` we defined earlier. Barriers are included to show the visual separation between the query gate implementation and the rest of the circuit, but they aren't necessary and can safely be removed.

```
def compile_circuit(function: QuantumCircuit):
    """
    Compiles a circuit for use in Deutsch's algorithm.
    """
    n = function.num_qubits - 1
    qc = QuantumCircuit(n + 1, n)

    qc.x(n)
    qc.h(range(n + 1))

    qc.barrier()
    qc.compose(function, inplace=True)
    qc.barrier()

    qc.h(range(n))
    qc.measure(range(n), range(n))

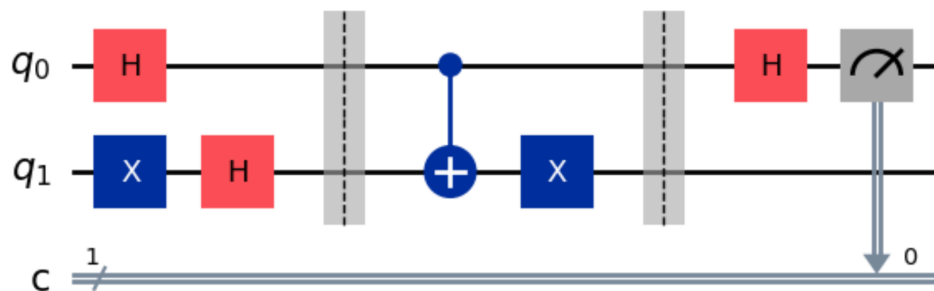
    return qc
```

✓ 0.0s

Again we can see what the circuit looks like using the draw method.

```
display(compile_circuit(deutsch_function(3)).draw(output='mpl'))
```

✓ 0.2s



Finally, we'll create a function that runs the circuit just defined one time and outputs the appropriate result: "constant" or "balanced."

```

from qiskit_aer import AerSimulator

def deutsch_algorithm(function: QuantumCircuit):
    """
    Determine if a Deutsch function is constant or balanced.
    """
    qc = compile_circuit(function)

    result = AerSimulator().run(qc, shots=1, memory=True).result()
    measurements = result.get_memory()
    if measurements[0] == "0":
        return "constant"
    return "balanced"

```

✓ 0.7s

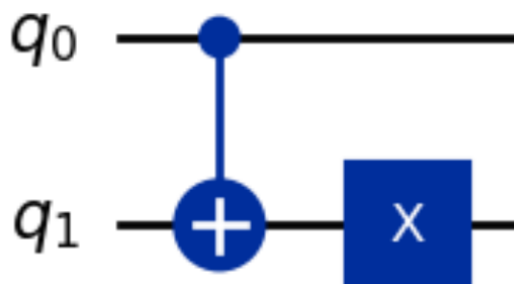
The following code cell runs Deutsch's algorithm on any one of the four functions defined above.

```

f = deutsch_function(3)
display(f.draw(output='mpl'))
● display(deutsch_algorithm(f))

```

✓ 0.1s



'balanced'